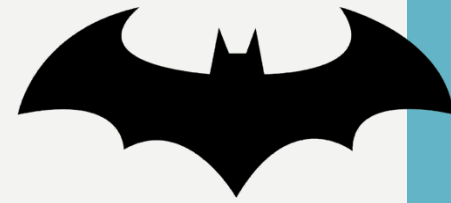




TP1 - LE SYSTÈME DE SÉCURITÉ DE LA BATCAVE

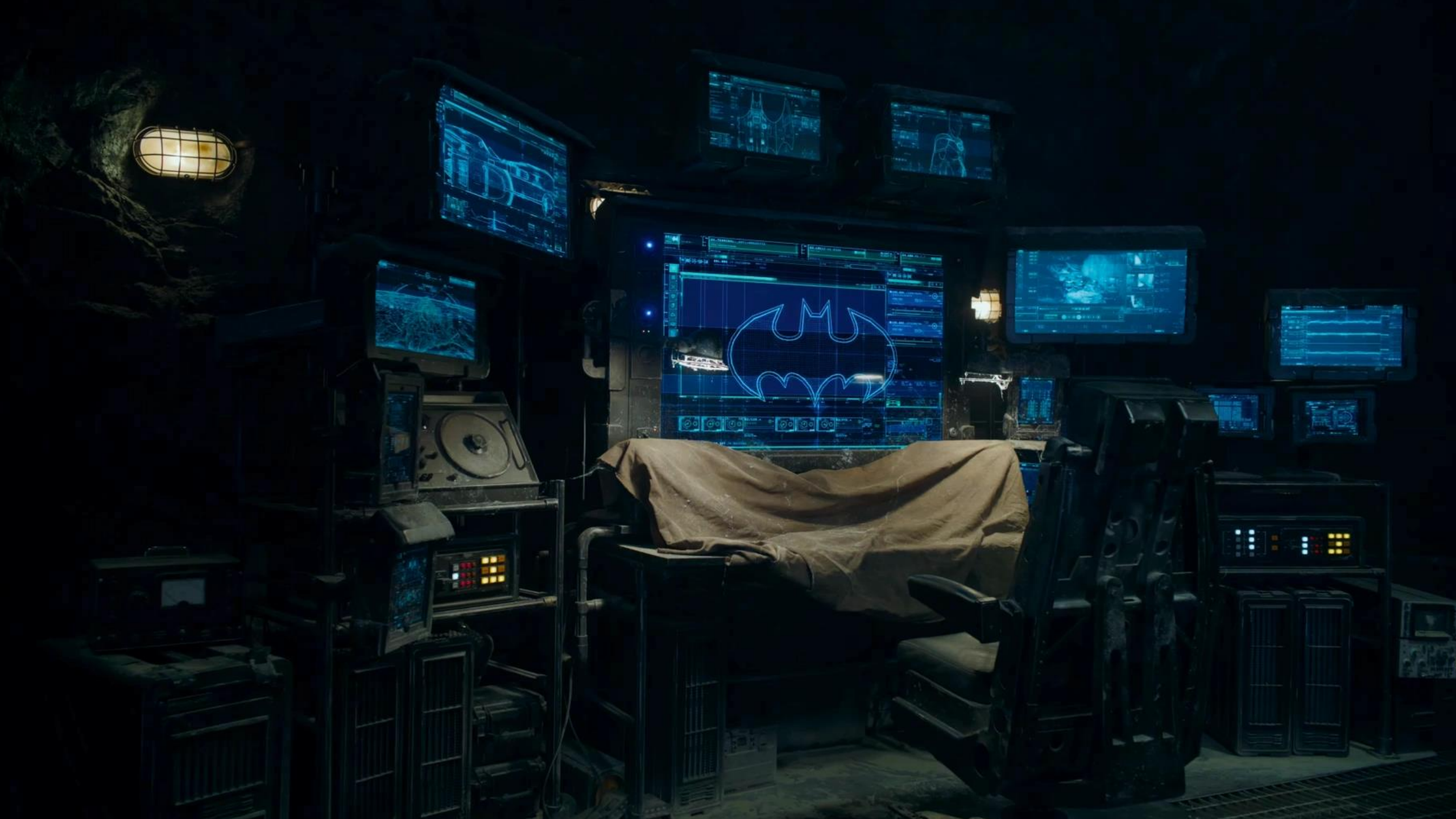
PROTOCOLES D'AUTHENTIFICATION

CONTEXTE ET OBJECTIF



Bruce Wayne a besoin de **moderniser les accès informatiques de la Batcave**. Alfred vous a donc missionné pour **créer un serveur centralisé gérant l'accès au "Bat-Ordinateur"**.

👉 Implémentez un système d'authentification **Basic Auth** sécurisé par **Bcrypt** et stocké en **SQLite**.



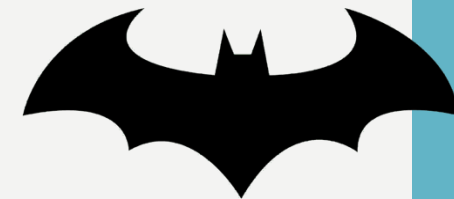


INSTRUCTIONS

CHRIS CHEVALIER

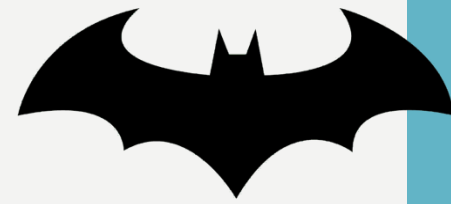
CHEVALIER@CHRIS-FREELANCE.COM

CONSEILS POUR RÉUSSIR



- Utilisez l'extension VS Code [SQLite Viewer](#) pour vérifier vos données et hachages de mot de passe.
- Pensez à utiliser la commande `node --watch server.js` pour ne pas perdre de temps. Vous pouvez configurer un script `"start"` dans le fichier `package.json` pour pouvoir utiliser la commande `npm start`.
- ⚠ N'oubliez pas le `return` dans votre middleware après un `res.status(401)` !

PHASE 1 : INITIALISATION



 *Ne vous reposez pas sur les outils d'IA générative, utilisez les documentations disponibles sur le web et le support de cours partagé.*

1. Créez un dossier `batcave-security`.
2. À l'intérieur, initialisez un projet Node.js.
3. Installez les dépendances nécessaires : `express`, `bcrypt` et `better-sqlite3`.
4. Configurez votre fichier `server.js` et votre base de données `database.db`. La BDD doit contenir, à minima, une **table** `users` avec un `id` (PK), un `username` (text unique) et un `password` (text).

PHASE 2 : LE REGISTRE DES JUSTICIERS

Avant d'entrer dans le système, il faut y être enregistré !

- **Côté Serveur** : Créez une **route** `POST /register` qui reçoit un `username` et un `password` dans le corps de la requête. Le mot de passe doit être **haché** avant d'être inséré dans la table `users`.
- **Côté Front (Public)** : Créez un fichier `public/register.html` avec un **formulaire** pour permettre à un nouveau justicier de **s'inscrire**. Vous pouvez utiliser un **framework CSS** pour le style. 💡 N'oubliez pas de **traiter les données saisies** par le justicier dans un fichier `public/register.js`.

PHASE 2.1 : ROBUSTESSE DU REGISTRE

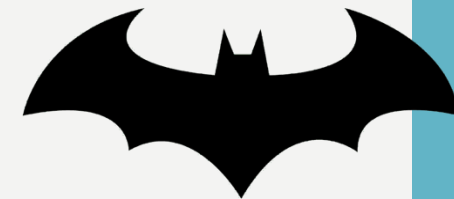


Un système de sécurité ne doit pas accepter n'importe quoi !

Améliorez votre **route** `POST /register`:

- Refusez l'inscription si le **mot de passe** ne fait pas 8 caractères ou plus ;
- Si un utilisateur tente de s'inscrire avec un **nom d'utilisateur déjà utilisé**, renvoyez un **code de statut** `409 Conflict` au lieu de laisser le serveur crasher à cause de la contrainte SQL `UNIQUE` ;
- Assurez-vous que le nom d'utilisateur ne contient **pas d'espaces** (utilisez la méthode JS `.trim()`).

PHASE 3 : PROTECTION DU BAT-ORDINATEUR



⚠ Le Bat-Ordinateur contient des fichiers sensibles ! Il faut sécuriser le **serveur** :

- Implémentez une **fonction middleware** qui :
 - Vérifie la présence de l'**en-tête** `Authorization` ;
 - **Décode** le Base64 ;
 - **Compare** les identifiants saisis avec ceux stockés en base ;
 - Renvoie une **réponse** `401` avec l'**en-tête** `WWW-Authenticate` en cas d'erreur.
- **Routes protégées** :
 - `GET /bat-computer` doit servir un fichier HTML `private/bat-computer.html` (⚠ **hors du dossier** `public`) ;
 - `GET /api/secrets` doit renvoyer un tableau d'objets au format JSON où chaque objet représente un des gadgets de Batman (ex: `{ "name": "Batarang", "desc": "Arme de jet", "icon": "fa-shuriken" }`).

PHASE 3.1 : PERSONNALISATION DE L'ACCÈS

Le **middleware** ne sert pas qu'à bloquer, il sert aussi à **identifier**. Améliorez l'expérience dans la Batcave :

- Créez une nouvelle **route** `GET /api/me` qui utilise les informations stockées dans `req.user` (par votre middleware) pour renvoyer, en **JSON**, le **nom de l'utilisateur authentifié** et son `id`.
- Modifiez le fichier `private/bat-computer.html`. Au chargement de la page, utilisez `fetch()` pour appeler la route `GET /api/me` et affichez un message personnalisé : `"Bienvenue, Justicier [Nom d'utilisateur]"`.
- Créez une route `POST /api/reports` permettant d'enregistrer une note de mission.
 - Le rapport doit être enregistré dans une **nouvelle table SQLite** `reports` et être **lié à l'id** de **l'utilisateur** qui l'envoie (clé étrangère).

PHASE 3.2 : L'ARSENAL DYNAMIQUE



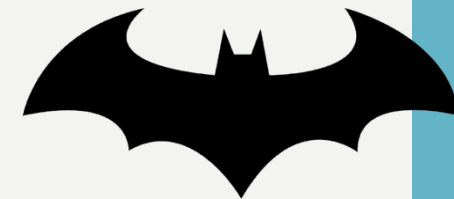
- **Côté client** (`private/bat-computer.html`) : Au chargement de la page, utilisez `fetch()` pour appeler votre route `GET /api/secrets`. 💡 N'oubliez pas d'injecter manuellement l'en-tête `Authorization: Basic <base64>` dans les options du `fetch()`.

👉 `headers.set('Authorization', 'Basic ' + Buffer.from(username + ":" + password).toString('base64'));`

- 🎨 **Rendu Graphique (DOM) : Bouclez sur les données** ainsi reçues pour générer dynamiquement des [cartes Bootstrap](#), par exemple. L'arsenal doit s'afficher sous forme de grille (utilisez le **standard CSS Flexbox** ou [Grid](#)).

⚠️ **Rappel de sécurité : Le navigateur ne gère pas l'en-tête de manière automatique** lors d'un appel `fetch()`. Si vous oubliez de configurer l'objet `headers` dans votre front-end, votre requête sera rejetée avec un code `401 Unauthorized`.

PHASE 4 : VALIDATION



Pour valider ce premier TP, vous devez réussir à vous connecter via 3 méthodes différentes :

1. **Via un navigateur web** : Accédez à la route `/bat-computer` et remplissez la boîte de dialogue native ;
2. **Via le logiciel [Postman](#)** : Téléchargez-le puis, utilisez l'onglet "Auth", sélectionnez "Basic Auth" et entrez vos identifiants. Observez l'en-tête généré ;
3. **Via l'interface en ligne de commande cURL** : Tentez d'accéder à la route `api/secrets` via votre terminal.



```
curl -u "votre_pseudo:votre_pass" http://localhost:3000/api/secrets
```

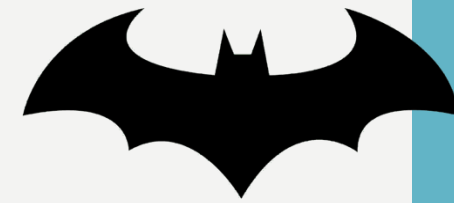


POUR ALLER PLUS LOIN

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

PHASE BONUS



Seulement pour les étudiants ayant terminé les phases précédentes !

Ajoutez une colonne `role` dans votre `table users`. Seuls les utilisateurs ayant le rôle `'ADMIN'` peuvent accéder aux `routes protégées`. Les autres reçoivent un message `"Accès refusé" (403)`.

Créez une `nouvelle table logs` dans `SQLite`. À chaque fois qu'un utilisateur parvient à se connecter à une `route protégée`, insérez une `ligne` avec son `username` et l'horodatage (un `timestamp`).

Implémentez un `compteur` en mémoire. Si un `utilisateur se trompe 3 fois de mot de passe`, bloquez ses tentatives pendant `30 secondes`.

Tentez d'implémenter une route `/logout` qui "force" la fermeture de la session.

A decorative element on the left side of the slide consisting of two parallel, wavy vertical lines. The inner line is light blue and the outer line is white, both set against the dark blue background.

DES QUESTIONS ?

CHEVALIER@CHRIS-FREELANCE.COM